# Smart Recipe Ontology (SRO)

Phase 2 Report

| Field | Value |
|---|---|
| Project | Smart Recipe Ontology (SRO) |
| Authors | Aleyna Gültekin 220315013, Esra Sepik 220315015, Sinem Tanrıver 220315073 |
| Course deliverable | Ontology Development Project - Phase 2 |
| Ontology version | 2.0 |
| Specification version | 2.0 |
| Prepared date | 08.05.2026 |
| GitHub repository | https://github.com/EsraSepik/ontology-design-project |
| Widoco documentation | https://esrasepik.github.io/ontology-design-project/ |

## Executive Summary

This Phase 2 report extends the Phase 1 Smart Recipe Ontology (SRO), which modeled recipes, ingredients, preparation steps, diet information, calories, and cooking time. The extended version adds a data acquisition pipeline, schema-guided ontology population, nutrition and provenance modeling, reusable ontology alignment, versioning, competency-query validation, and documentation instructions.

| Phase 2 requirement | What is included in this report |
|---|---|
| Research integration | Primary selected study: Ontology Population using LLMs. It is integrated into SRO as the semi-automatic recipe-to-triples population pipeline. |
| Mandatory paper review | A review of Personalized Ontology and Deep Training Tree-Based Optimal GRU-RNN for Prediction of Students Behavior, including methodology, contributions, and adaptation to SRO. |
| Data acquisition | Sources, collection method, preprocessing, quality control, and mapping from raw recipe fields to SRO classes and properties. |
| Ontology usage and expansion | Existing vocabularies, new classes/properties, modeling methodology, and validation queries. |
| GitHub update | Repository structure and version-control commands. Repository URL must be filled after upload. |
| Widoco documentation | Widoco commands and documentation URL placeholder. A lightweight preview is included, but official Widoco output should replace it before final submission. |
| Specification document v2 | A separate v2 ORSD/specification document is included in the submission package. |

## 1. Phase 1 Baseline

The Phase 1 ORSD defined SRO as a structured knowledge model for cooking recipes. It focused on recipes, ingredients, preparation steps, dietary information, calories, and cooking time. Its competency questions asked for vegetarian/vegan recipes, recipes using a specific ingredient, ingredients used in a recipe, calories, cooking duration, required kitchen utilities, sequential preparation steps, and gluten-free recipes.

| Original scope item | Phase 2 extension |
|---|---|
| Recipe structure | Kept as the core class sro:Recipe and aligned with schema:Recipe. |
| Ingredients | Extended through sro:IngredientUsage to represent quantity, unit, state, and extraction confidence. |
| Preparation steps | Extended with ordered sro:PreparationStep instances, step numbers, nextStep/precedesStep relations, and required utilities. |
| Diet and calories | Extended with controlled diet individuals, nutrition profiles, allergen metadata, and per-serving nutrient values. |
| Querying | Validated with SPARQL competency queries included in queries/competency_queries.rq. |

## 2. Research Integration: Ontology Population using LLMs

### 2.1 Selected study

The selected Week 11 study is Ontology Population using LLMs by Norouzi, Barua, Christou, Gautam, Eells, Hitzler, and Shimizu. The paper investigates whether large language models can extract structured triples from natural-language text when the output is constrained by a modular ontology schema. The study reports that, when a modular ontology is included as guidance in the prompts, LLM extraction can approach approximately 90% triple coverage against ground truth.

### 2.2 Why this study was selected

- SRO needs to populate a recipe knowledge graph from semi-structured or unstructured recipe text, not only manually create classes in Protege.
- Recipes naturally contain text fields: ingredient lines, cooking instructions, quantities, units, times, and diet descriptions. These are exactly the kinds of fields that require extraction and mapping to ontology concepts.
- The paper gives a practical strategy for reducing hallucination: provide the ontology modules as a schema constraint, retrieve only relevant text, ask the model to skip unsupported facts, and validate the output before loading triples.

### 2.3 Relevance to SRO

The core relevance is ontology population. In Phase 1, SRO was mainly a conceptual model. In Phase 2, SRO must also explain how data will enter the ontology. The selected study is used to design the SRO data acquisition pipeline: recipe sources are collected, cleaned, segmented into modules, mapped with an ontology-guided prompt, converted to RDF, and validated through SPARQL/SHACL-style checks.

| Paper idea | SRO adaptation |
|---|---|
| Modular ontology guidance in prompts | Provide Recipe, IngredientUsage, PreparationStep, NutritionProfile, DietType, and Utility modules to the extraction prompt. |
| Data preprocessing before extraction | Clean recipe pages or CSV rows, normalize ingredient lines, remove irrelevant text, deduplicate recipes, and segment instructions into steps. |
| Text retrieval and summarization | Extract only ingredient, instruction, time, cuisine, nutrition, and diet fragments from long recipe documents before generating triples. |
| KG population | Generate TTL/CSV triples such as Recipe -> hasIngredientUsage -> IngredientUsage -> usesIngredient -> Ingredient. |
| Evaluation using ground truth similarity | Compare generated triples with manually checked sample recipes and run competency SPARQL queries. |

### 2.4 Integration point in SRO architecture

The integration is planned in the data processing and ontology population layer, before reasoning/querying:

1. Data source connector: retrieves recipe JSON/CSV/manual documents.

2. Preprocessor: cleans text, normalizes times and units, splits ingredient and instruction fields.

3. Ontology-guided extractor: uses SRO modules as schema constraints and produces candidate triples.

4. Validation layer: checks required fields, domain/range expectations, datatype formats, diet consistency, and provenance.

5. RDF loader: stores accepted triples in the ontology file or a triplestore.

6. Reasoning/query layer: answers competency questions and future user questions through SPARQL.

Example extraction prompt pattern planned for SRO:

```
Input: recipe title, ingredient lines, instructions, nutrition text.
Schema: Recipe, IngredientUsage, Ingredient, PreparationStep, NutritionProfile, DietType.
Task: output only facts supported by the input. Use the format subject | predicate | object.
Rule: if a quantity, unit, diet tag, or calorie value is missing, do not invent it; mark unknown for
manual review.
```

# 3. Mandatory Paper Review

## 3.1 Paper reviewed

Mandatory paper: Personalized ontology and deep training tree-based optimal gated recurrent unit-recurrent neural network for prediction of students behavior by F. Mary Harin Fernandez, T. Venkata Ramana, Mahammad Shabana, V. Kannagi, and M. Nalini. The paper proposes a personalized digital library framework that combines ontology engineering in Protege 4.3 with a GRU-RNN model enhanced by Deep Training Tree (DTT) and Black Widow Optimization (BWO).

## 3.2 Core methodology

1. Collect digital library usage and student behavior information such as profile, browsing history, course interaction, learning objects, and feedback.

2. Preprocess behavioral and textual data, including stop-word elimination and preparation of browsing/search behavior features.

3. Create the digital library ontology in Protege 4.3 through domain knowledge acquisition, ontology organization, ontology elaboration, consistency checking, and ontology validation.

4. Organize the ontology into sub-ontologies such as learner performance, digital library operation, course planning, digital library, digital library analysis, and learner behavior.

5. Use reasoning and inference rules to support learner profile assignment and recommendations.

6. Train a GRU-RNN model with DTT to predict cognitive behavior, learning speed behavior, sedentary behavior, and aggressive behavior.

7. Use BWO to update GRU-RNN weights and address delayed convergence and over-fitting.

8. Evaluate the system with accuracy, precision, recall, F-measure, loss, and 10-fold cross-validation.

## 3.3 Key contributions

| Contribution | Explanation |
|---|---|
| Ontology-based personalization | The study models personalized digital library services through ontology structures, user profiles, sub-ontologies, and interrelated concepts. |
| Hybrid symbolic and neural system | Ontology reasoning supports structured recommendation, while GRU-RNN predicts user behavior traits. |
| Deep Training Tree | DTT is introduced to reduce the vanishing-gradient problem associated with GRU-RNN training. |
| Black Widow Optimization | BWO tunes GRU-RNN weights to improve convergence and reduce over-fitting. |
| Measured performance | The paper reports 0.92 accuracy in 10-fold cross-validation, approximately 97.4% F-measure, 98.2% recall, and 97.5% precision. |

## 3.4 Adaptation to SRO

A full GRU-RNN behavior prediction module is outside the current Phase 2 scope because SRO does not yet have a large user-behavior dataset. However, the paper is still useful for SRO because it shows how ontology-based personalization can be separated from a later predictive layer.

| Paper component | Possible SRO adaptation |
|---|---|
| User profile ontology | Add sro:UserPreference for diet restrictions, disliked ingredients, cuisine preference, calorie target, cooking skill, and time limit. |
| Reasoning rules | Use ontology rules/SPARQL filters to recommend recipes matching diet, time, calories, allergens, and available ingredients. |
| Behavior prediction | Future extension: collect viewed/saved/rejected recipes and train a sequence model only after enough user-interaction data exists. |
| Feedback loop | Record whether a user accepts a recipe recommendation and use this to update preference scores. |

# 4. Data Acquisition Section

## 4.1 Data sources

| Source | Use in SRO | Fields expected | Status |
|---|---|---|---|
| Manual validation CSV | Small trusted sample for testing the ontology and SPARQL queries. | title, ingredients, steps, time, diet tags, nutrition, source | Included as data/sample_recipe_data_phase2.csv |
| TheMealDB API | Public recipe API source for recipe names, categories, areas/cuisines, instructions, ingredients, and measurements. | meal name, category, area, instructions, ingredients, measures, image/source fields | Planned external source |
| RecipeNLG dataset | Large-scale recipe text corpus for future ontology population experiments. | title, ingredients, directions, source text | Planned dataset source; check license before redistribution |
| USDA FoodData Central API | Nutrition lookup source for ingredient nutrient values and calories. | food identifier, nutrient values, food details | Planned nutrition source |
| Instructor-provided or team-curated documents | Controlled source for manually verified examples and edge cases. | recipe text, ingredient lines, step sequence, diet labels | Recommended for grading reliability |

## 4.2 Data collection methodology

1. Start with a manually verified sample dataset of 10-20 recipes to test modeling choices and competency queries.

2. Collect additional recipe records from a recipe API or dataset only after checking access and licensing conditions.

3. Store each raw record unchanged in a raw-data folder, then create a cleaned normalized CSV for ontology mapping.

4. Record provenance for every recipe using sro:derivedFromSource and sro:sourceUrl or a local source identifier.

5. Assign a validation status: raw, extracted, automatically validated, or manually validated.

## 4.3 Preprocessing steps

| Step | Description | Output |
|---|---|---|
| Deduplication | Normalize recipe title and source URL; remove duplicate records. | Unique recipe list |
| Language filtering | Keep English records for the current SRO version because Phase 1 non-functional requirements specify English support. | English-only recipe records |
| Ingredient parsing | Split ingredient lines into quantity, unit, ingredient name, and preparation state. | IngredientUsage candidates |
| Unit normalization | Map unit strings such as g, gram, tbsp, cup, piece to controlled SRO/QUDT unit identifiers where possible. | Normalized units |
| Step segmentation | Split instructions into ordered preparation steps and assign stepNumber. | PreparationStep instances |

| | | |
|---|---|---|
| Diet and allergen tagging | Map tags such as vegan, vegetarian, gluten-free and derive allergen hints from ingredient categories. | DietType and Allergen annotations |
| Nutrition enrichment | Use trusted nutrition data when available; otherwise leave missing values unknown and mark for manual review. | NutritionProfile values |
| Validation | Check required properties, datatypes, domain/range, duplicate step numbers, and contradictions. | Accepted or review-needed triples |

### 4.4 Mapping data to ontology concepts

| Raw data field | Ontology class/property | Example mapping |
|---|---|---|
| recipe_id | Individual IRI for sro:Recipe | R001 -> sro:MediterraneanChickpeaSalad |
| title | rdfs:label on sro:Recipe | "Mediterranean Chickpea Salad" |
| cuisine | sro:hasCuisine -> sro:Cuisine | Mediterranean -> sro:MediterraneanCuisine |
| course | sro:hasCourseCategory -> sro:CourseCategory | Salad -> sro:SaladCourse |
| diet_tags | sro:suitableForDiet -> sro:DietType and recipe subclass typing | vegan -> sro:VeganDiet and type sro:VeganRecipe |
| prep/cook/total time | sro:prepTimeMinutes, sro:cookTimeMinutes, sro:totalTimeMinutes | 10, 0, 10 |
| ingredient line | sro:IngredientUsage with sro:usesIngredient, sro:quantityValue, sro:unitLabel | 240 g chickpeas -> quantity 240, unit gram, ingredient chickpea |
| steps | sro:PreparationStep with sro:stepNumber, sro:instructionText, sro:nextStep | Step 1 -> Dice cucumber and tomatoes. |
| nutrition | sro:NutritionProfile and nutrient data properties | 250 kcal -> sro:caloriesPerServing 250 |
| source | sro:derivedFromSource -> sro:DataSource | manual test data -> sro:ManualTestData |
| confidence/status | sro:extractionConfidence and validation status in data table | 1.0 for manually validated sample triples |

## 5. Ontology Usage and Expansion

### 5.1 Existing ontologies/vocabularies used

| Ontology/vocabulary | Source | How SRO uses it |
|---|---|---|
| Schema.org Recipe | schema.org | SRO aligns sro:Recipe with schema:Recipe and reuses the modeling idea of recipeIngredient, recipeInstructions, nutrition, cookTime, recipeCuisine, recipeCategory, and suitableForDiet. |
| Schema.org NutritionInformation | schema.org | SRO aligns sro:NutritionProfile with schema:NutritionInformation for calories and nutrient values. |
| QUDT | qudt.org | SRO uses QUDT-style unit modeling and maps SRO units such as gram and milliliter to QUDT unit identifiers. |
| Dublin Core Terms / PROV style provenance | W3C/semantic web vocabularies | SRO includes creator/version metadata and source provenance for recipes. |

### 5.2 Expansion methodology

The expansion follows a hybrid METHONTOLOGY plus Ontology Design Pattern approach. METHONTOLOGY is used for specification, conceptualization, formalization, implementation, evaluation, and maintenance. ODP-style modeling is used for recurring modeling problems: n-ary ingredient quantities, ordered step sequences, controlled diet categories, and provenance.

| Method step | Action in SRO v2 |
|---|---|
| Specification | Updated competency questions and non-functional requirements from Phase 1. |
| Conceptualization | Identified new concepts: IngredientUsage, NutritionProfile, DietType, Allergen, Cuisine, CourseCategory, CookingMethod, DataSource, and UserPreference. |
| Formalization | Defined OWL classes, object properties, datatype properties, domains, ranges, labels, comments, and sample individuals. |
| Implementation | Implemented the ontology in Turtle and RDF/XML serializations. |
| Evaluation | Executed SPARQL competency queries over the example data and included query_results.txt. |
| Maintenance | Versioned as v2.0 and prepared GitHub tag and Widoco documentation instructions. |

### 5.3 New or extended ontology elements

| Element type | Examples added in v2 |
|---|---|
| Classes | Recipe, VegetarianRecipe, VeganRecipe, GlutenFreeRecipe, Ingredient, IngredientUsage, PreparationStep, NutritionProfile, DietType, Cuisine, CourseCategory, CookingMethod, KitchenUtility, Unit, DataSource, UserPreference. |
| Object properties | hasIngredientUsage, usesIngredient, hasPreparationStep, nextStep, precedesStep, requiresUtility, hasNutritionProfile, suitableForDiet, hasCuisine, hasCourseCategory, hasCookingMethod, hasAllergen, hasUnit, derivedFromSource. |
| Datatype properties | quantityValue, unitLabel, preparationState, stepNumber, instructionText, prepTimeMinutes, cookTimeMinutes, totalTimeMinutes, servings, caloriesPerServing, proteinGrams, fatGrams, carbohydrateGrams, sourceUrl, extractionConfidence, fdcId. |
| Example individuals | MediterraneanChickpeaSalad, VegetableOmelette, ChickenRiceBowl, Chickpea, Egg, ChickenBreast, VeganDiet, GlutenFreeDiet, MixingBowl, Pan, Pot. |

### 5.4 Competency query validation

| CQ | SPARQL validation status |
|---|---|
| CQ1: Which recipes are vegetarian or vegan? | Validated: returns Mediterranean Chickpea Salad and Vegetable Omelette. |
| CQ2: Which recipes use a specific ingredient? | Validated: chickpea returns Mediterranean Chickpea Salad. |
| CQ3: What ingredients are used in a recipe? | Validated: returns ingredient, quantity, and unit for Mediterranean Chickpea Salad. |
| CQ4: How many calories does a recipe contain? | Validated: returns calories per serving for all example recipes. |
| CQ5: How long does it take to cook a recipe? | Validated: returns prep, cook, and total time. |
| CQ6: Which kitchen utilities are required? | Validated: returns utilities such as knife, cutting board, mixing bowl, pan, and pot. |
| CQ7: What are the sequential steps? | Validated: ordered stepNumber values are returned. |
| CQ8: Which recipes are gluten-free? | Validated: returns Mediterranean Chickpea Salad and Vegetable Omelette. |

## 6. GitHub Update and Version Control

The ontology and report files are prepared for GitHub upload. The repository owner must upload them because account access is required.

```
git add ontology data queries docs reports README.md
git commit -m "Phase 2: extend SRO ontology, data acquisition, research integration, and specification v2"
git tag v2.0
git push origin main --tags
```

Repository placeholder to replace before submission: <GITHUB_REPOSITORY_URL>

## 7. Widoco Documentation

The ontology file is ready for Widoco. Run one of the following commands from the repository root after uploading the ontology.

```
docker run -ti --rm -v "$PWD/ontology:/usr/local/widoco/in" -v "$PWD/docs:/usr/local/widoco/out"
dgarijo/widoco -ontFile in/smart-recipe-ontology-v2.ttl -outFolder out -rewriteAll -webVowl

java -jar widoco.jar -ontFile ontology/smart-recipe-ontology-v2.ttl -outFolder docs -rewriteAll -webVowl
```

Documentation URL placeholder to replace before submission: <WIDOCO_DOCUMENTATION_URL>

## 8. Specification Document v2 Change Summary

| Area | Version 1 | Version 2 change |
|---|---|---|

| | | |
|---|---|---|
| Purpose/scope | Focused on recipe structure, ingredients, steps, diet, calories, cooking time. | Expanded with data acquisition, ontology population, provenance, nutrition profiles, and validation. |
| Functional requirements | Eight original competency questions. | Original questions retained and mapped to SPARQL; new data/provenance and personalization-related requirements added. |
| Data model | Recipe, Ingredient, Step, Utility, Unit, Cuisine. | Added IngredientUsage, NutritionProfile, DietType, Allergen, CourseCategory, CookingMethod, DataSource, UserPreference. |
| Research basis | Not included. | Integrated Ontology Population using LLMs and reviewed mandatory GRU-RNN paper. |
| Documentation | Basic ORSD. | Versioned ontology, report, spec v2, query files, and Widoco instructions. |

## 9. References


1. Gultekin, A., Sepik, E., and Tanriver, S. Smart Recipe Ontology Phase 1 ORSD, 2026.

2. Norouzi, S. S., Barua, A., Christou, A., Gautam, N., Eells, A., Hitzler, P., and Shimizu, C. Ontology Population using LLMs, 2024.

3. Fernandez, F. M. H., Venkata Ramana, T., Shabana, M., Kannagi, V., and Nalini, M. Personalized ontology and deep training tree-based optimal gated recurrent unit-recurrent neural network for prediction of students behavior. Concurrency and Computation: Practice and Experience, 2023.

4. Schema.org. Recipe and NutritionInformation vocabulary documentation.

5. TheMealDB. Free Recipe API documentation.

6. USDA FoodData Central. API Guide.

7. QUDT. Quantity, Unit, Dimension and Type ontology documentation.

8. Garijo, D. WIDOCO: Wizard for Documenting Ontologies documentation.